



//Cap Contracts PR 273 Review

September 18 2026

About Octane

Founded in 2023 and headquartered in San Francisco, Octane delivers a truly holistic security solution through continuous code reviews that protect protocols throughout their entire development lifecycle. We combine software-first vulnerability analysis with world-class manual security talent, enabling teams to ship faster while maintaining the highest security standards. Our clients including Circle (USDC), Uniswap, Avalanche, Plume, and Sophon trust us to protect \$5.8B+ in assets because we catch vulnerabilities that traditional assessments miss.

Clients choose Octane for:

- Our software-first approach: We've built a platform trained on thousands of real-world vulnerabilities that consistently identifies complex exploit patterns traditional methods miss. Every commit and pull request receives automatic review through native CI/CD integration, with vulnerabilities identified through deep dependency analysis and exploit path tracing.
- Our novel research: Octane's security researchers who've collectively earned over \$1M in bug bounties provide comprehensive manual reviews on an as-needed basis, maximizing code security and developing further insights for automated protection.
- Our proven track record preventing exploits: We've discovered 100+ vulnerabilities across 86M+ lines of code that were missed in traditional security reviews. We prevented exploits similar to Rikkei Finance (\$1.1M), caught complex slippage bugs in Uniswap integrations, identified oracle mispricing vulnerabilities, and found assembly errors that evaded static analyzers. Our platform adapts to your team's preferences, dramatically reducing false positives as the platform learns your codebase while providing exact code modifications and exploit scenarios for immediate remediation.
- Our commitment to ecosystem security: We partner directly with blockchain foundations to establish network-wide security baselines. Integration requires no code changes and takes minutes to deploy, after which teams receive continuous protection with rapid response. We offer flexible engagement from basic reviews to enterprise coverage with 24/7 support and formal security reports, supporting all EVM-compatible chains with developing Rust capabilities.

Through our holistic approach, we transform security from a development bottleneck into an enabler of velocity and confidence. We live by iterative development and in August FY25 alone, we boosted accuracy on challenging vulnerability classes by 31% and increased our detection coverage by 12.7%. To learn how continuous security accelerates development while protecting users, visit [octane.security](https://www.octane.security).

1. Executive Summary

PR 273 adds a 100% APR limit to the restaker rate stored by `RateOracle`. It also adds a transient-storage reentrancy guard to the five primary cUSD vault accounting functions: `mint`, `burn`, `redeem`, `borrow`, and `repay`.

The review identified one informational-severity finding. The new rate check applies only to future calls to `setRestakerRate`. Because the oracle is upgraded through an existing UUPS proxy, a restaker rate above 100% that was stored before the upgrade remains in namespaced proxy storage and continues to accrue without being capped.

No actionable issue was identified in the transient reentrancy guard integration. The guard does not alter persistent proxy storage and does not require initialization. The new tests successfully block reentry into each guarded accounting function from a malicious ERC-4626 fractional-reserve callback.

Findings Summary

ID	Title
CAP-I01	Pre-upgrade over-cap restaker rates are not normalized or rejected during the oracle upgrade

2. Scope and Coverage

The review assessed [PR 273](#) of the [cap-labs-dev/cap-contracts](#) repo, merging `fix/limit-restaker-rate` into `dev`:

- base commit: `07c15e06a4479d193e82760a1d790d31b8254a33`;
- head commit: `224d7c593769fe58feaf3ac2a2f38e51528353e3`; and

The files changed by the PR were:

- `contracts/interfaces/IRateOracle.sol`;
- `contracts/oracle/RateOracle.sol`;
- `contracts/vault/Vault.sol`;
- `snapshots/Vault.gas.t.json`;
- `test/mocks/MockERC4626.sol`;
- `test/mocks/MockReentrantERC4626.sol`; and
- `test/vault/Vault.reentrancy.t.sol`.

The review followed these changes into the oracle proxy, namespaced oracle storage, restaker interest calculation, EigenLayer and Symbiotic agent managers, vault accounting libraries, fractional-reserve logic, deployment access control, and infrastructure upgrade tests where those components determined the security behavior of the PR.

The review focused on:

- correctness and bypass resistance of the 100% restaker-rate limit;
- persistence of existing rates across a UUPS oracle upgrade;
- rate units, arithmetic, and boundary behavior;
- direct and forwarded rate-setting paths;
- persistent storage compatibility after adding the reentrancy guard;
- EIP-1153 target-chain support and transient-state initialization;
- modifier placement and cross-function reentrancy coverage;
- ERC-20 and ERC-4626 callback surfaces; and
- access-controlled and inherited vault entry points.

3. System Overview

3.1 Restaker-rate model

`RateOracle` stores one `restakerRate` for each agent. Rates use ray precision: `1e27` represents 100% APR. Authorized callers can update the rate directly through `setRestakerRate` or indirectly through the EigenLayer and Symbiotic agent managers.

When an agent's rate changes through either manager, the manager previews interest accrued under the old rate for each configured reserve and realizes it when the preview is nonzero. It then writes the new rate to the oracle. If the oracle rejects the new rate, the entire transaction reverts, including any preceding interest realization. When old-rate accrual is zero or rounds to zero, the manager skips realization and does not advance `lastRealizationTime`; that pre-existing checkpoint behavior is outside this PR's changes.

The lending pool reads the stored rate when calculating accrued restaker interest. The calculation is linear for one accrual interval. A stored value of `1e27` represents a nominal 100% APR, while a value above `1e27` represents a nominal rate above 100%. Realized interest is added to the DebtToken balance used by later accrual calculations, so periodic realization can produce an effective annual percentage yield above the nominal APR. PR 273 caps the configured nominal rate; it does not cap compounded APY or total debt growth from other interest components.

3.2 Oracle upgrade and storage model

`Oracle` is a UUPS-upgradeable implementation. Its rate data is held by the proxy in an ERC-7201-style namespaced storage location derived from `cap.storage.RateOracle`.

Upgrading the implementation preserves the `restakerRate` mapping. The upgrade does not call `setRestakerRate` for existing agents and PR 273 does not add a reinitializer or migration step. New validation in the setter therefore governs future writes but does not validate values already present in proxy storage.

3.3 Vault and fractional reserves

The cUSD vault tracks backing supplies and agent borrows. Idle backing can be invested in an external ERC-4626 fractional-reserve vault. Burns, redemptions, and borrows may withdraw assets from that external vault before completing cUSD accounting and transfers.

These ERC-4626 calls create an external callback surface. A malicious or compromised fractional-reserve vault can call back into cUSD while the outer operation is incomplete. ERC-20 transfers and `transferFrom` calls create additional callback surfaces for non-standard tokens.

3.4 Transient reentrancy guard

PR 273 makes `Vault` inherit OpenZeppelin's `ReentrancyGuardTransientUpgradeable` and applies `nonReentrant` to `mint`, `burn`, `redeem`, `borrow` and `repay`.

The guard uses EIP-1153 transient storage. Its lock is scoped to the vault address and the current transaction, resets when the transaction ends, and does not occupy persistent proxy storage. The zero value means “not entered,” so the guard does not require any initialization.

4. Findings

4.1 CAP-I01: Pre-upgrade over-cap restaker rates are not normalized or rejected during the oracle upgrade

Severity: Informational

Description

PR 273 rejects future restaker-rate writes above `1e27`. Before this change, `setRestakerRate` accepted any `uint256`. An existing oracle proxy can therefore contain a rate above `1e27`.

The UUPS upgrade preserves that value in the `restakerRate` mapping. PR 273 does not enumerate existing agents, migrate their rates, or enforce the limit when a rate is read. The lending pool continues to use the stored value directly in `accruedRestakerInterest`.

As a result, an agent whose rate was already above `1e27` continues accruing more than 100% APR after the upgrade. The new setter check takes effect only if an authorized account later updates that agent's rate.

Impact

A legacy over-cap rate can continue overcharging restaker interest after the patched implementation is deployed. The effect persists until an administrator writes a compliant rate.

Exploitation requires an over-cap value to have been stored before the upgrade. Rate setters are access-controlled, and the review found no unprivileged path to create a new over-cap value after the upgrade.

Recommendation

Enumerate every configured agent before upgrading and inspect its current `restakerRate`. Reset any value above `1e27` before or atomically with the implementation upgrade.

5. Security analysis and integration considerations

5.1 The new write boundary is correct

The setter accepts `1e27` and rejects any larger value. This matches the documented ray units and the linear annual interest calculation. No alternate production setter or direct storage-writing path was identified.

The EigenLayer and Symbiotic agent managers forward rate changes to the same oracle setter. When their old-rate preview is nonzero, they realize that interest before writing the new rate; a rejected write then reverts the entire transaction. There is no partial realization caused by the new cap.

If the old-rate preview is zero or rounds to zero, both managers skip realization and leave `lastRealizationTime` unchanged. A later positive rate can therefore apply across the stale interval. This behavior predates PR 273 and is not a bypass of the new `1e27` write limit, but it should be tracked and tested separately.

5.2 Upgrade procedures remain part of the rate invariant

CAP-I01 exists because implementation checks do not retroactively validate proxy state. The upgrade runbook must treat the current mapping contents as part of the deployment input.

A pre-upgrade script should list all registered agents, read each current rate, fail if any value exceeds `1e27`, and save the checked block number. If remediation and upgrade occur in separate transactions, monitor rate changes between the check and execution or perform both through one governed batch.

5.3 The transient guard preserves persistent storage layout

`ReentrancyGuardTransientUpgradeable` does not add ordinary storage fields to the vault proxy. Adding it to the inheritance list therefore does not shift the vault's persistent storage or its existing namespaced state.

The guard reads a transient boolean whose default value is false at the start of a transaction, sets it while a protected call executes, and clears it afterward. A revert unwinds the transient write, so failed access checks or downstream calls do not leave the vault locked.

5.4 The guarded accounting surface blocks the tested callback class

All five primary balance-changing accounting entry points share one lock. A callback entered from a guarded `burn`, `redeem`, or `borrow` path therefore cannot switch to another guarded function.

The PR's tests configure a malicious ERC-4626 vault that calls back during divestment. They verify rejection of nested `mint`, `burn`, `redeem`, `borrow`, and `repay` calls. This covers the cross-function behavior the shared guard is meant to prevent.

5.5 Inherited fractional-reserve entry points remain outside the lock

`investAll`, `divestAll`, `setFractionalReserveVault`, and `realizeInterest` can invoke an external ERC-4626 vault without first acquiring the transient lock. A callback originating from one of these functions can therefore enter a guarded accounting function because the lock is still clear.

No concrete permissionless asset-loss or accounting-corruption path was established from this condition. The relevant administrative functions are access-controlled, and nested accounting operations still require the callback caller's tokens, allowances, or borrow and repay authorization. The configured reserve vault is also already trusted enough to deny withdrawals or misreport ERC-4626 behavior.

Still, applying the same guard to every state-changing function that calls the configured ERC-4626 vault would reduce the callback surface and make the protection easier to reason about. Add tests with callbacks originating from `invest`, `full divest`, `vault replacement`, and `interest realization` before extending modifier coverage.

6. Verification

6.1 Properties reviewed

The review examined:

- ray units and the exact 100% boundary;
- all direct and forwarded restaker-rate write paths;
- interest realization before a rate transition;
- arithmetic use of the stored restaker rate;
- UUPS upgrade behavior and persistence of namespaced rate storage;
- reentrancy through ERC-4626 withdraw, redeem, deposit, and view callbacks;
- reentrancy through ERC-20 transfer and `transferFrom` calls;
- cross-function behavior of the shared transient lock;
- inherited vault and fractional-reserve entry points;
- access control on borrow, repay, and reserve-management functions;
- persistent storage compatibility after changing inheritance; and
- EIP-1153 support for the evidenced deployment target.

No route was found to write a new over-cap rate after the upgrade without causing the transaction to revert. No persistent storage collision, stuck transient lock, or concrete asset-loss path was identified in the reentrancy changes.

6.2 Existing test suite

Suite	Result
<code>VaultReentrancyTest</code>	5 passed
<code>RateOracleGetRateTest</code>	3 passed
<code>TestInfraUpgrade</code>	2 passed

6.3 Additional verification recommended

Add regressions for:

- a direct `setRestakerRate` call at exactly `1e27`;
- a direct call at `1e27 + 1` that expects `RateTooHigh`;
- EigenLayer and Symbiotic forwarded calls at both boundaries;
- zero-to-positive, dust-to-positive, and positive-to-positive rate changes, including whether `lastRealizationTime` advances when old-rate accrual is zero;

- a pre-upgrade rate above `1e27` retained through an oracle proxy upgrade;
- effective post-upgrade accrual when legacy storage is above the limit;
- callbacks initiated by `investAll`, `divestAll`, `setFractionalReserveVault`, and `realizeInterest`; and
- deployment or execution checks on every target chain where the guarded vault implementation may run.